DAY 77 | PAGINATION | SPRING DATA PAGEABLE | DTO WRAPPER & REDIS

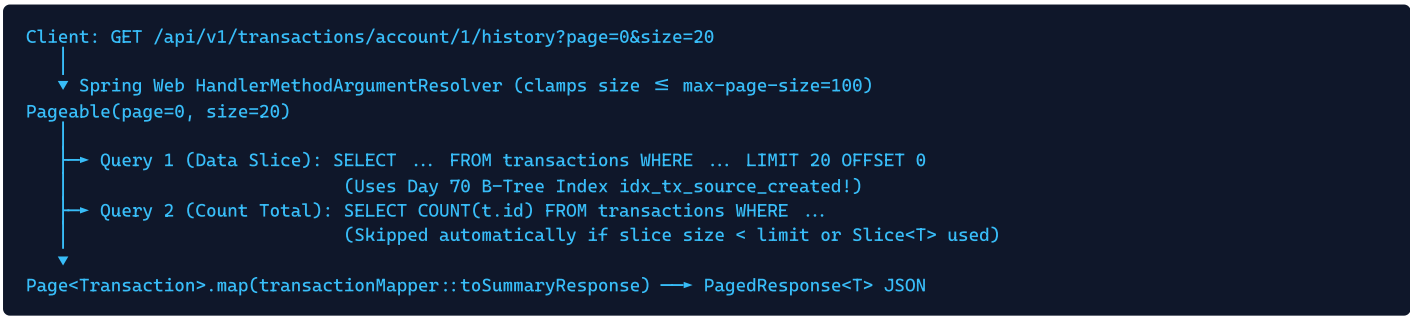
Pagination & Pageable Banking APIs

Study Notes & Planned Implementation Blueprint: Bounding Unbounded List Queries with Spring Data Pageable, DoS Page-Size Limits, Jackson 3 / Redis-Safe PagedResponse<T>, and Day 70 Composite Index Alignment.

PROJECT	SPRING DATA	RESPONSE SHAPE	MAX PAGE CAP
Enterprise Digital Banking	**Commons & JPA 4.1.0**	**PagedResponse<T>**	**100 Records / Call**

1. Core Concepts: Why Unbounded `List<T>` Endpoints Fail at Scale

Today, every list endpoint in our codebase (`TransactionController.getTransactionHistory`, `AuditController.getAllAuditLogs`, `LedgerController.getLedgerByAccount`, `CustomerController.getAllCustomers`, etc.) returns a full `List<T>`. With 10 rows in development it feels fast; with 500,000 ledger entries in production, a single `GET /history` loads half a million entities into JVM heap, triggers GC pauses, saturates network bandwidth, and crashes clients.



`Page<T>` vs `Slice<T>` vs `List<T>`

- `List<T>`: No SQL `LIMIT`, dangerous for unbounded tables.
- `Page<T>`: Executes **2 SQL queries** (data slice + `COUNT(*)`). Best for admin tables needing total pages & jump-to-page UI.
- `Slice<T>`: Requests `size + 1` rows to set `hasNext` with **NO `COUNT(*)`** query! Best for infinite mobile scroll feeds.

Why Custom `PagedResponse<T>` Over Raw `Page<T>`?

- Spring Data 4 warns when serializing raw `PageImpl` directly to JSON: internal structure is unstable across versions.
- `RedisCacheConfig` cannot safely serialize/deserialize raw `PageImpl` (used by `AccountServiceImpl`!).
- A lean `PagedResponse<T>` exposes an explicit, clean JSON API contract & Swagger schema.

2. Codebase Audit: Endpoints to Convert from `List<T>` to `PagedResponse<T>`

Module / Controller	Endpoint	Growth Risk	Day 77 Design Decision
TransactionController	`GET /api/v1/transactions/account/{accountId}/history`	CRITICAL	Primary target! Default `size=20`, mapped via `TransactionSummaryResponse`. Also used by `load-test.ps1` (returns `200 OK`).
LedgerController	`GET /api/v1/ledger/account/{accountId}`	CRITICAL	Double-entry ledger grows 2x per transfer! Paginate with `size=20` using `idx_ledger_account_created`.
AuditController	`GET /api/v1/audit`, `/entity/...`, `/user/...`	HIGH	Audit logs grow infinitely; all 3 list methods must accept `Pageable`.
CustomerController	`GET /api/v1/customers`	HIGH	Calls `customerRepository.findAll(pageable)` directly instead of `findAll()`!
AccountController	`GET /api/v1/accounts/customer/{customerId}`	MEDIUM	Update Redis `@Cacheable` key in `AccountServiceImpl` to include page/size/sort!
Loan / Beneficiary / EMI	`/loans/customer/{id}`, `/beneficiaries/customer/{id}`, `/emis/loan/{id}`	LOW-MED	Standardize every collection API on `PagedResponse<T>` for uniform client SDKs.

> **Important Note on `LedgerController.getLedgerByTransaction`:** A single transaction produces at most **2 ledger entries** (1 DEBIT + 1 CREDIT). Keeping `/ledger/transaction/{transactionId}` as a small bounded list (or paginated if standardized) is worth noting, while `/ledger/account/{accountId}` MUST be paginated.

3. Planned Code: Universal `PagedResponse<T>` & Safeguard Properties

src/main/resources/application.properties (Pagination DoS Safeguards) MODIFIED

```
# ═══════════════════════════════════════════════════════════════════
# Spring Data Web Pagination Safeguards (Day 77)
# ═══════════════════════════════════════════════════════════════════
spring.data.web.pageable.default-page-size=20
spring.data.web.pageable.max-page-size=100
spring.data.web.pageable.one-indexed-parameters=false
spring.data.web.pageable.page-parameter=page
spring.data.web.pageable.size-parameter=size
```

> **DoS Protection:** Setting `max-page-size=100` stops any attacker or buggy frontend from calling `?size=9999999` to crash JVM memory. Spring MVC automatically caps oversized requests down to `100`.

src/main/java/ ... /common/dto/PagedResponse.java NEW

```java
package org.example.enterprisedigitalbankingsystem.common.dto;

import lombok.*;
import org.springframework.data.domain.Page;
import java.io.Serializable;
import java.util.List;

@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class PagedResponse<T> implements Serializable {

    private static final long serialVersionUID = 1L;

    private List<T> content;
    private int page;
    private int size;
    private long totalElements;
    private int totalPages;
    private boolean first;
    private boolean last;
    private boolean empty;

    public static <T> PagedResponse<T> from(Page<T> springPage) {
        return PagedResponse.<T>builder()
                .content(springPage.getContent())
                .page(springPage.getNumber())
                .size(springPage.getSize())
                .totalElements(springPage.getTotalElements())
                .totalPages(springPage.getTotalPages())
                .first(springPage.isFirst())
                .last(springPage.isLast())
                .empty(springPage.isEmpty())
                .build();
    }
}
```

4. Planned Code: Repositories (Spring Data `Pageable` Queries)

TransactionRepository.java, LedgerRepository.java & AuditRepository.java MODIFIED

```java
// 1. TransactionRepository.java
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;

public interface TransactionRepository extends JpaRepository<Transaction, Long> {
    Optional<Transaction> findByTransactionReference(String transactionReference);
    boolean existsByTransactionReference(String transactionReference);

    Page<Transaction> findBySourceAccountIdOrDestinationAccountId(
            Long sourceAccountId, Long destinationAccountId, Pageable pageable);
}

// 2. LedgerRepository.java
public interface LedgerRepository extends JpaRepository<LedgerEntry, Long> {
```

```java
    Page<LedgerEntry> findByAccountId(Long accountId, Pageable pageable);
    List<LedgerEntry> findByTransactionId(Long transactionId);
}

// 3. AuditRepository.java
public interface AuditRepository extends JpaRepository<AuditLog, Long> {
    Page<AuditLog> findAll(Pageable pageable);
    Page<AuditLog> findByEntityNameAndEntityId(String entityName, String entityId, Pageable pageable);
    Page<AuditLog> findByPerformedBy_UserId(UUID userId, Pageable pageable);
}

// 4. AccountRepository, LoanRepository, BeneficiaryRepository, EmiRepository
Page<Account> findByCustomerId(Long customerId, Pageable pageable);
Page<Loan> findByCustomerId(Long customerId, Pageable pageable);
Page<Beneficiary> findByCustomerId(Long customerId, Pageable pageable);
Page<Emi> findByLoanId(Long loanId, Pageable pageable);
```

5. Planned Code: Services & Controllers

TransactionService.java, TransactionServiceImpl.java & TransactionController.java — MODIFIED

```java
// TransactionService.java
PagedResponse<TransactionSummaryResponse> getTransactionHistory(Long accountId, Pageable pageable);

// TransactionServiceImpl.java
@Override
@Transactional(readOnly = true)
public PagedResponse<TransactionSummaryResponse> getTransactionHistory(Long accountId, Pageable pageable) {
    if (!accountRepository.existsById(accountId)) {
        throw new ResourceNotFoundException("Account not found with id: " + accountId);
    }
    Page<TransactionSummaryResponse> mapped = transactionRepository
            .findBySourceAccountIdOrDestinationAccountId(accountId, accountId, pageable)
            .map(transactionMapper::toSummaryResponse);
    return PagedResponse.from(mapped);
}

// TransactionController.java
@GetMapping("/account/{accountId}/history")
public ResponseEntity<PagedResponse<TransactionSummaryResponse>> getTransactionHistory(
        @PathVariable @Positive Long accountId,
        @PageableDefault(size = 20, sort = "createdAt", direction = Sort.Direction.DESC) Pageable pageable) {
    return ResponseEntity.ok(transactionService.getTransactionHistory(accountId, pageable));
}
```

AccountServiceImpl.java (Redis Cache Key Harmonization) & Summary DTO Serialization — MODIFIED

```java
// Ensure AccountSummaryResponse implements Serializable so RedisCacheManager handles PagedResponse cleanly!
@Override
@Transactional(readOnly = true)
@Cacheable(value = "accountsByCustomer",
           key = "#customerId + ':' + #pageable.pageNumber + ':' + #pageable.pageSize + ':' + #pageable.sort.toString()")
public PagedResponse<AccountSummaryResponse> getAccountsByCustomerId(Long customerId, Pageable pageable) {
    if (!customerRepository.existsById(customerId)) {
        throw new ResourceNotFoundException("Customer not found with id: " + customerId);
    }
    Page<AccountSummaryResponse> page = accountRepository.findByCustomerId(customerId, pageable)
            .map(accountMapper::toSummaryResponse);
    return PagedResponse.from(page);
}
```

Micro-Optimization Spotted in Service Impl: Notice in `TransactionServiceImpl.getTransactionHistory`, calling `accountRepository.existsById(accountId)` instead of `accountRepository.findById(accountId)` issues a lightweight `SELECT count/1` instead of hydrating the full `Account` + joining `Customer`!

CustomerController, AuditController, LedgerController, LoanController, BeneficiaryController, EmiController — MODIFIED

```java
// CustomerController.java
@GetMapping
public ResponseEntity<PagedResponse<CustomerSummaryResponse>> getAllCustomers(
        @PageableDefault(size = 20, sort = "createdAt", direction = Sort.Direction.DESC) Pageable pageable) {
    return ResponseEntity.ok(customerService.getAllCustomers(pageable));
}
```

```java
// AuditController.java
@GetMapping
public ResponseEntity<PagedResponse<AuditLogResponse>> getAllAuditLogs(
        @PageableDefault(size = 20, sort = "createdAt", direction = Sort.Direction.DESC) Pageable pageable) {
    return ResponseEntity.ok(auditService.getAllAuditLogs(pageable));
}

// LedgerController.java
@GetMapping("/account/{accountId}")
public ResponseEntity<PagedResponse<LedgerEntryResponse>> getLedgerByAccount(
        @PathVariable @Positive Long accountId,
        @PageableDefault(size = 20, sort = "createdAt", direction = Sort.Direction.DESC) Pageable pageable) {
    return ResponseEntity.ok(ledgerService.getLedgerByAccount(accountId, pageable));
}
```

6. Verification & Performance Notes

- **Verify Pagination Metadata**: `curl -H "Authorization: Bearer $TOKEN" "http://localhost:8080/api/v1/transactions/account/1/history?page=0&size=5"` — verify response contains `content` (5 items), `totalElements`, `totalPages`, `first: true`, `last`.
- **Verify DoS Clamp**: Request `?size=5000` and verify returned `size` is clamped strictly to `100`!
- **Interview Deep Dive — Offset vs Keyset/Cursor Pagination**: Standard `OFFSET 100000` still requires MySQL to scan and discard 100,000 index entries. At multi-million row scale (e.g., bank reconciliation), replace deep offsets with *Keyset/Cursor pagination* (`WHERE created_at < :cursor ORDER BY created_at DESC LIMIT 20`), which achieves constant `O(log N)` latency on any page depth using Day 70's composite index!